Programa:	DVD Region+CSS Free 5.9.6.8
Protección:	ASProtect 1.2x - 1.3x [Registered] + Nagscreen + Trial 30 días
Objetivos:	Registrarnos por el jeto, jeje..
URL descarga:	http://www.dvdidle.com
Dificultad:	Súper fácil!
Herramientas:	OllyDbg v1.10 + PEiD 0.94 + PEditor v1.7
Cracker:	SyXe'05 ... [hAcK-c0d3d] ... [CracksLatinoS] Fecha: 26-01-06

Hola a todos nuevamente ;-)

Bueno lo prometido es deuda así que allá vamos con la segunda parte del tute, en la primera vimos cómo obtener un desempakado del programa mediante un script, nos quedó el archivo tute_.exe con el que podemos trabajar perfecto, mirémoslo con PEiD 0.94:

Microsoft Visual C++ 6.0

Está compilado en Visual C++ 6.00 esto es bueno saberlo, sobretodo porque según el compilador nos encontraremos unas u otras estructuras, no es lo mismo C++ que Delphi ni VBasic, etc., cada uno tiene sus características propias.

Esta segunda parte tratará de evitar las limitaciones y/o restricciones propias del programa sin nada que ver ya con el ASProtect, el packer no está ya presente y por lo tanto no controla nada del proceso, así que todo lo que veamos debemos saber que es del mismo programa en sí.

Dicho esto vamos al ataque, cuando corremos el programa nos muestra esta nagscreen:

Nos da variar posibilidades, entre ellas está la de ingresar un serial, también seguir con el período Trial (Evaluación) y es aquí donde están las restricciones, si introducimos un serial el programa se cierra, seguramente intentará validarlo cuando inicia el programa, quizá se guarde el serial en el registro o en algún archivo y luego al arrancar lo recupere mediante llamadas a alguna API, de todos modos por aquí no atacaremos, iremos más fácil todavía.

Lanzando un ataque, jeje…

Lo que haremos será lo siguiente: cuando esté la nag en pantalla pausamos Olly con F12, miramos en la pila y allí veremos diferentes direcciones de memoria con un "Return to:" al lado, esto significa que hubo una llamada justo antes de hay un CALL desde el cual se lanza la nag, si miramos en la pila vemos dos llamadas un tanto sospechosas:

```
0012F174  00000000
0012F178  00406A85  RETURN to tute_.00406A85 from tute_.0040F300
0012F17C  00000000
0012F180  0012F1A8
0012F184  0042A378  tute_.0042A378
0012F188  00000000
0012F18C  0012F16C
0012F190  0012FDA8  Pointer to next SEH record
0012F194  73DE3801  SE handler
0012F198  00000000
0012F19C  0012FDB4
0012F1A0  00404148  RETURN to tute_.00404148 from <JMP.&mfc42.#2514>
0012F1A4  00423FE8  tute_.00423FE8
0012F1A8  004194E0  tute_.004194E0
```

Son las dos que resalté en amarillo, nos situamos sobre la línea de la primera de ellas y pulsamos Enter para caer en la parte de código de esta llamada, allí ponemos un BP:

```
Address   Hex dump        Disassembly                          Comment
00406A85  . 83A6 F80B000( AND DWORD PTR DS:[ESI+BF8],0
00406A8C  . 8B4D F4       MOV ECX,DWORD PTR SS:[EBP-C]
00406A8F  . 5F            POP EDI
00406A90  . C706 E094410( MOV DWORD PTR DS:[ESI],tute_.004194E0
00406A96  . 8BC6          MOV EAX,ESI
00406A98  . 5E            POP ESI
00406A99  . 5B            POP EBX
00406A9A  . 64:890D 0000( MOV DWORD PTR FS:[0],ECX
00406AA1  . C9            LEAVE
00406AA2  . C2 0400       RETN 4
```

Vamos a la otra dirección y hacemos exactamente lo mismo:

```
Address   Hex dump        Disassembly                          Comment
00404148  . 83A6 E801000( AND DWORD PTR DS:[ESI+1E8],0
0040414F  . 834D FC FF    OR DWORD PTR SS:[EBP-4],FFFFFFFF
00404153  . 8D8D F4F3FFF( LEA ECX,DWORD PTR SS:[EBP-C0C]
00404159  . E8 0D000000   CALL tute_.0040416B
0040415E  > 8B4D F4       MOV ECX,DWORD PTR SS:[EBP-C]
00404161  . 5E            POP ESI
00404162  . 64:890D 0000( MOV DWORD PTR FS:[0],ECX
00404169  . C9            LEAVE
0040416A  . C3            RETN
```

Y volvemos a dejar correr el programa con F9, no saltará ninguno de los BP's puestos pero no pasa nada por esto, pulsamos en el botón donde dice "Evaluación gratuita" y enseguida paramos en uno de ellos, concretamente en el segundo, caemos en :404148 y si somos observadores veremos en la ventana de registros un bonito EAX==0, esto le indica el programa que hemos pulsado este botón, interesante, jeje.. pulsamos Ctrl+F9 para llegar al siguiente RETN y lo superamos para caer en esta parte:

```
Address   Hex dump        Disassembly                          Comment
00403693  . 8BC8          MOV ECX,EAX
00403695  . E8 7BFBFFFF   CALL tute_.00403215                  ojo a este call..
0040369A  . 85C0          TEST EAX,EAX
0040369C  .v 75 20        JNZ SHORT tute_.004036BE             bonito salto ;-)
0040369E  . 8BCE          MOV ECX,ESI
004036A0  . C786 B402000( MOV DWORD PTR DS:[ESI+2B4],1
004036AA  . E8 3A0A0000   CALL tute_.004040E9                  aqui se llama a la nag!
004036AF  . 391D E005420( CMP DWORD PTR DS:[4205E0],EBX
004036B5  .v 74 07        JE SHORT tute_.004036BE
004036B7  > 33F6          XOR ESI,ESI
004036B9  .v E9 D3000000  JMP tute_.00403791
004036BE  > 53            PUSH EBX
004036BF  . 8BCE          MOV ECX,ESI
```

Vaya vemos que la llamada a la nagscreen se realizó en :4036AA justo antes de donde estamos ahora y un poco antes un CALL con un TEST EAX,EAX sospechoso porque si tuviéramos que EAX vale 1 por ejemplo se tomaría el salto y no pasaríamos por el call que muestra la nag, esto nos hace pensar que en ese call ocurre algo importante, nos aseguraremos de ello, le ponemos un BP y reiniciamos Olly:

```
Address   Hex dump        Disassembly                          Comment
00403693  . 8BC8          MOV ECX,EAX
00403695  . E8 7BFBFFFF   CALL tute_.00403215                  ojo a este call..
0040369A  . 85C0          TEST EAX,EAX
0040369C  .v 75 20        JNZ SHORT tute_.004036BE             bonito salto ;-)
0040369E  . 8BCE          MOV ECX,ESI
```

Pulsamos F9 y caemos en él, entramos con F7…

```
Address   Hex dump        Disassembly                          Comment
00403215  $ E8 E6DDFFFF   CALL tute_.00401000
0040321A  . 05 04010000   ADD EAX,104
0040321F  .v 74 0F        JE SHORT tute_.00403230
00403221  . 50            PUSH EAX
00403222  . E8 891B0100   CALL <JMP.&msvcrt.strlen>            $ strlen
00403227  . 85C0          TEST EAX,EAX
00403229  . 59            POP ECX
0040322A  .v 76 04        JBE SHORT tute_.00403230
0040322C  . 33C0          XOR EAX,EAX
0040322E  . 40            INC EAX
0040322F  . C3            RETN
00403230  > 33C0          XOR EAX,EAX
00403232  . C3            RETN
```

Esto es lo que vemos muy poco código en realidad, quitamos el anterior BP y ponemos uno aquí, volvemos a reiniciar Olly y pulsamos F9, al momento estamos aquí, entramos en la primera llamada donde sólo se hace esto:

```
Address   Hex dump    Disassembly                    Comment
00401000  $ B8 E0334200  MOV EAX,tute_.004233E0
00401005  . C3           RETN
```

Simplemente coloca el valor 4233E0 en EAX y retorna así que aquí hay poco que hacer, después llega una llamada a strlen que se utiliza para contar los caracteres de una cadena pasada como parámetro (el push anterior) y justamente este PUSH es la dirección pasada a EAX en el call anterior:

```
Address   Hex dump    Disassembly                    Comment
00403215  $ E8 E6DDFFFF  CALL tute_.00401000
0040321A  . 05 04010000  ADD EAX,104
0040321F  . 74 0F        JE SHORT tute_.00403230
00403221  . 50           PUSH EAX                   s = ""
00403222  . E8 891B0100  CALL <JMP.&msvcrt.strlen>  strlen
00403227  . 85C0         TEST EAX,EAX
```

Si seleccionamos EAX y hacemos un "Follow in dump/Memory address" veremos que allí no hay nada:

```
Address   Hex dump                                        ASCII
004234E4  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
004234F4  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00423504  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00423514  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00423524  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

Allí pondremos nuestro nombre a ver que pasa:

```
Address   Hex dump                                        ASCII
004234E4  53 79 58 65 27 30 35 00 00 00 00 00 00 00 00 00  SyXe'05.........
004234F4  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00423504  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00423514  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00423524  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

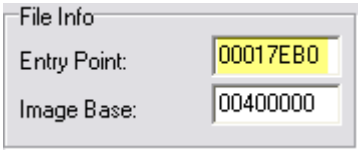
Pulso F9 y el programa arranca sin la nag ¿? Jeje se lo ha tragado? Voy al about y veo:

Vaya nos hemos registrado¡! Así de sencillo? Está claro que confiaban en ASProtect, jeje.. porque esto no es una protección ;-)

El parche simplemente será injertar la llamada para que escriba nuestro nombre en la dirección que marca EAX, es tan sencillo como buscar una zona llena de ceros en el ejecutable y ensamblar un par de líneas, la zona libre la tenemos en :417EB0 por ejemplo, vamos allí y escribimos esto:

```
Address   Hex dump    Disassembly                    Comment
00417EB0  C705 E434420(  MOV DWORD PTR DS:[4234E4],65587953
00417EBA  C705 E834420(  MOV DWORD PTR DS:[4234E8],353027
00417EC4  68 464E4100    PUSH tute_.<ModuleEntryPoint>
00417EC9  C3             RETN
```

Lo que hace es escribir mi nombre en la posición [4234E4] donde después el programa lo leerá y si lo encuentra ZAS! Estamos registrados, salvamos los cambios "Copy to executable/Selection" y "Save file" después, lo llamamos tute_crk.exe, ahora cogemos el .exe injertado y le cambiamos el EP hacia el injerto para que arranque desde allí y modifique esa posición de memoria, esto lo haremos con PEditor que va muy bien para estas tareas, hacerlo es sencillo: abrimos el exe y cambiamos donde pone EP de 00014E46 a 00017EB0 que es el nuevo EntryPoint:



salvamos los cambios, lo probamos y… Quedamos registrados, jeje..

Una buena práctica para hacer injertos para los que se están iniciando, un tute sencillo la verdad que esperaba algo mejor pero bueno así son las cosas no las elijo yo. Para cualquier cosa ya sabéis donde encontrarme:

syxe00@yahoo.es
syxe05@gmail.com

Un saludo y un abrazo a todo el grupo SIN EXCEPCIÓN!!!